COLORADO
Information Security
Governor's Office of Information Technology

TS-APP-005: Secure Application Software Standards and Configuration Management

Document ID	TS-APP-005
Title	Secure Application Software Standards & Configuration Management
Executive Sponsor	Chief Information Security Officer (CISO)
Issuing Authority	Governor's Office of Information Technology (OIT)
Applies to	Internal development, third-party software (COTS/SaaS), and all architecture layers (web, mobile, API, microservices, serverless) across Sandbox, Development, UAT, Staging, and Production environments.
Tier	Standard
Version	3.0
Effective Date	11/19/2015
Last Updated	04/02/2026

1. Introduction

1.1 Purpose

The purpose of this standard is to mandate a consistent set of security requirements across the software development lifecycle (SDLC). By adhering to these controls, the organization minimizes the risk of data breaches, ensures regulatory compliance (e.g., GDPR, SOC2, HIPAA), and protects the integrity of business operations. It supports compliance with the Colorado Information Security Policies (CISPs).

1.2 Scope

This standard applies to:

- Internal Development: All source code authored by employees or contractors.
- Third-Party Software: Commercial off-the-shelf (COTS) products and SaaS integrations.
- Architecture: Web applications, mobile apps, APIs, microservices, and serverless functions.
- Environments: All stages of the pipeline, including Sandbox, Development, UAT, Staging, and Production.

1.3 Audience

- Engineering Teams: Developers and DevOps engineers responsible for implementation.
- Security Teams: Auditors and AppSec engineers validating compliance.
- Product Management: For the inclusion of security requirements in the product roadmap.

2. Guiding Principles

These principles form the philosophical foundation of the organization's security posture and should guide decision-making where specific technical rules may be ambiguous.

- Security by Design and Default: Security is a core requirement from the requirements-gathering phase. Systems must be “secure-by-default,” meaning the most restrictive settings are applied out of the box.
- Defense in Depth: Multiple independent layers of defense (e.g., WAF, authentication, encryption) must be implemented so the failure of one does not lead to total compromise.
- Principle of Least Privilege (PoLP): Users and system processes are granted only the minimum access levels necessary to perform their functions.
- Fail Securely: In the event of an error or system failure, the application must default to a secure state (e.g., denying access rather than allowing it).
- Assume Breach (Zero Trust): Operate under the assumption that the network is hostile; every request, internal or external, must be explicitly authenticated, authorized, and encrypted.
- Psychological Acceptability: Security controls should be as unobtrusive as possible so they do not impede productivity, which often leads to users seeking insecure workarounds.

3. Technical Control Requirements

3.1 Identity and Access Management (IAM)

- Standardized Protocols: Applications must use OIDC or SAML 2.0 for authentication via the corporate Identity Provider.
- Multi-Factor Authentication (MFA): Mandatory for all applications accessing sensitive data or administrative interfaces.
- Session Management: Secure cookie attributes required (HttpOnly, Secure, SameSite=Lax/Strict). Absolute session timeouts must not exceed 12 hours; inactivity timeouts must not exceed 30 minutes for sensitive applications.

3.2 Data Protection

- Encryption in Transit: Mandatory use of TLS 1.2 or greater for all data movement; legacy protocols (TLS 1.0/1.1, SSL) are prohibited.
- Encryption at Rest: All sensitive data in databases or file storage must be encrypted using AES-256.
- Secret Management: Credentials, API keys, and certificates must be stored in an approved Secret Management Vault. Hardcoding secrets in source code is a critical violation.

3.3 Application Integrity

- Input Validation: All data originating from an external source must be validated against a strict allow-list (type, length, and format).
- Output Encoding: Context-aware encoding must be applied to all user-supplied data before it is rendered in a browser to prevent Cross-Site Scripting (XSS).
- Injection Prevention: Parameterized queries (prepared statements) must be used for all database interactions.

4. Secure Development Lifecycle (SDLC)

Security activities shall be integrated into the automated CI/CD pipelines.

4.1 Requirements and Design

- Security Requirements: Defined and documented alongside functional requirements.
- Threat Modeling: Performed for all new applications and significant changes to existing applications to identify and mitigate potential threats.
- Secure Architecture and Design: The application architecture and design shall incorporate security best practices.
- Data Identification Assessment: Identify PII/sensitive data handling for proper controls and development configurations.

4.2 Development

- Secure Coding Practices: All developers shall be trained on and adhere to secure coding best practices.
- Input Validation: All input from untrusted sources shall be validated to prevent common vulnerabilities such as injection attacks and XSS.
- Output Encoding: All output to the user interface shall be properly encoded to prevent XSS.
- Dependency Management: All third-party libraries and components shall be from trusted sources, with a process in place to manage and update them.

4.3 Testing

- Static Application Security Testing (SAST): Integrated into the development pipeline across multiple phases at minimum — Coding Phase (IDE), Commit/Push Phase, Build Phase (CI/CD), and Release/Staging Phase.
- Dynamic Application Security Testing (DAST): Performed on running applications across multiple phases at minimum — QA/Staging Phase (Pre-Release) and Production Phase (Post-Release); the Production Phase check shall occur at least quarterly.
- Vulnerability Management: A process shall be in place to track, prioritize, and remediate all identified vulnerabilities as defined and directed by CISO policy.

4.4 Deployment and Operations

- Secure Configuration: Applications and their underlying infrastructure shall be securely configured.
- Logging and Monitoring: Sufficient logging and monitoring shall be in place to detect and respond to security incidents.
- Incident Response: An incident response plan shall be in place to address security incidents.

5. Enforcement and Exceptions

5.1 Compliance Monitoring

The Security Operations Center (SOC) will perform periodic audits and continuous monitoring. Non-compliant applications may be subject to immediate revocation of production access.

5.2 Exception Process

Exceptions to this standard must be documented in the Risk Register, include a business justification, and require formal sign-off from the Chief Information Security Officer (CISO).

6. Glossary

- Application: A software program that provides a specific function for a user or another application.
- Cross-Site Scripting (XSS): A type of injection attack in which malicious scripts are injected into otherwise benign and trusted websites.

- Dynamic Application Security Testing (DAST): A method of testing an application in its running state.
- Injection Attack: A type of attack that involves injecting malicious code into an application.
- Multi-Factor Authentication (MFA): A security process requiring more than one method of authentication from independent categories of credentials to verify a user's identity.
- Penetration Testing: An authorized simulated cyberattack on a computer system, performed to evaluate the security of the system.
- Static Application Security Testing (SAST): A method of testing an application by examining its source code.
- Threat Modeling: A process by which potential threats can be identified, enumerated, and prioritized from a hypothetical attacker's point of view.
- Vulnerability: A weakness in an application that can be exploited by a threat actor.